

To do list:

- Bio on the Account page needs to be removed.
- Have specific admin page when logged in using the admin credentials
 - Allows the admin to see all the journeys and make judgements - like the one in figma
- Potential feature, if the location has been entered in multiple times, it autofills it when the user logs in - can implement this as a typing and searching box and not just a typing one
- Get the visuals to a good stage so that they look nice
- Implement the matching Visuals

Our inputs and outputs:**- Inputs:**

- Email,
- Password,
- First Name
- Last Name
- Department
- Post Code
- Placed Marker on the map
- Departure Time,
- Method of transport
- Phone number

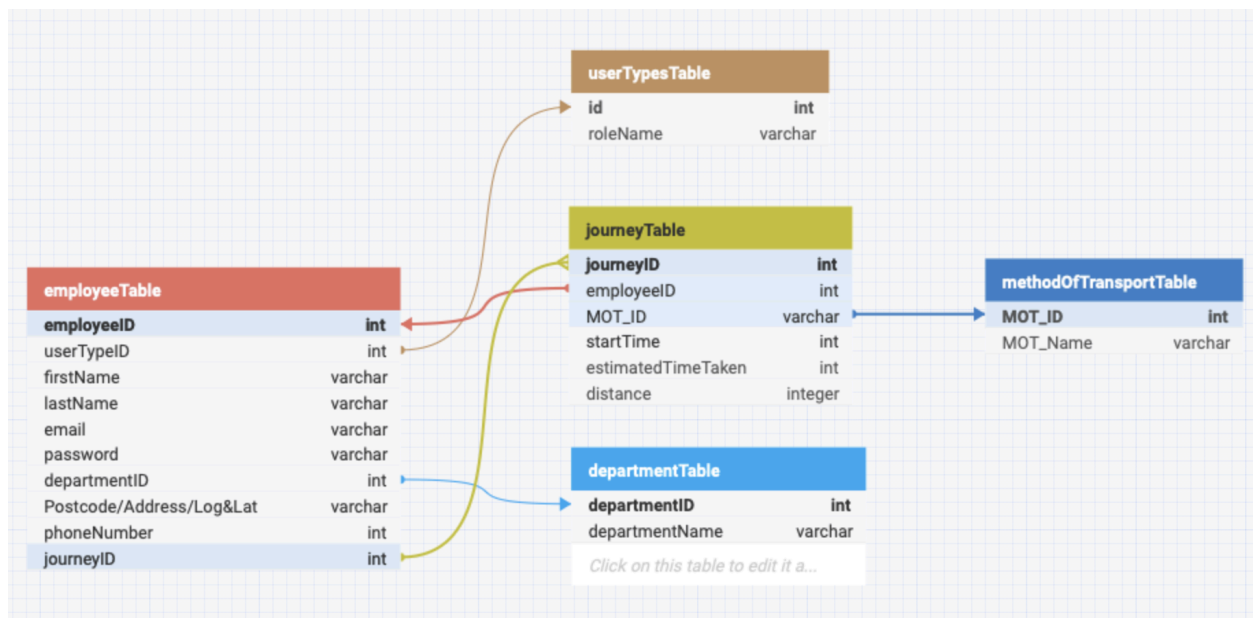
- Outputs:

- Exact longitude & Latitude
- Distance
- Estimated Time
- Estimated Carbon Emissions
 - The visuals that go with that
- The matching of two people.

- What we would like the database to do:

- To create a new user when they input all of their information on the sign up page - there must be a table with that information in it with some sort of foreign key to match the journeys to the user - look below for what the database should look like
- To fetch the users details and verify them to what they have inputted - this has been done already with the user verification
- To be able to output back user information for it to be displayed.
 - Employee ID, first and last name, method of transport, start time, emissions, phone number/ email (for contacts) and if they match.

- What we think the database should look like - it is also in 3NF, which is the best format a database can have:



- Notice how all of these tables are connected together with the different relationships as well and the data types are all added as well. Everything that can be simplified, has been as well'

The code to create it:

```
employeeTable {
    employeeID int pk
    userTypeID int > userTypesTable.id
    firstName varchar
    lastName varchar
    email varchar
    password varchar
    departmentID int > departmentTable.departmentID
    Postcode/Address/Log&Lat varchar
    phoneNumber int
    journeyID int > * journeyTable.journeyID
}

journeyTable {
    journeyID int pk
    employeeID int > employeeTable.employeeID
    MOT_ID varchar > methodOfTransportTable.MOT_ID
```

```
        startTime int
        estimatedTimeTaken int
        distance integer
    }

    userTypesTable {
        id int pk
        roleName varchar
    }

    departmentTable {
        departmentID int pk
        departmentName varchar
    }

    methodOfTransportTable {
        MOT_ID int pk increments unique
        MOT_Name varchar
    }
```

The Matching algorithm UI prototype - when you press find more, it takes you to that popup screen:

